这份报告并不说明任何键的真实状态，驱动必须原样保留上一份。若把它当成“全部松开”，就会先上报一串假松开，下一份报告再把同样的键当作新按下，同一个字符因此重复两遍。

之后的事由输入子系统（kernel/input.c、kernel/input.h）完成。类驱动只用到七个接口：

```
struct input_dev *input_allocate_device(void);
void input_set_capability(struct input_dev *dev, int type, int code);
int  input_register_device(struct input_dev *dev);
void input_report_key(struct input_dev *dev, int code, int value);
void input_sync(struct input_dev *dev);
void input_unregister_device(struct input_dev *dev);
void input_free_device(struct input_dev *dev);
```

probe() 用前三项登记设备，运行期只用中间两项上报状态变化，remove 时用后两项收尾；事件去重、按键状态、环形队列、阻塞读、SYN_REPORT 和 SYN_DROPPED 全部由 core 负责，usbkbd.c 里只剩 USB 与 HID 的职责。core 再把事件分发给两个“处理者”：

- **evdev**：/dev/input/event0。每次 open 都分配一个独立的客户端环形缓冲区（fops.open 设置 f->private_data），读者在自己的缓冲区上睡眠；溢出时插入 SYN_DROPPED；
- **控制台**：把按键转成字符（处理 Shift、Caps Lock、Ctrl），送入 consoleintr()。所以 USB 键盘和串口一样可以登录、输入命令、按 Ctrl-C。

这正是 Linux 的分层：硬件驱动只关心“产生了什么事件”，不关心“谁在读”。

内核侧的按键序列。 usbkbd_rx_work() 处理一份报告时的动作顺序值得摊开看，因为它同时喂了上面两个处理者：

1. 本次 report[2..7] 里有、previous[2..7] 里没有的 usage 是新按下：先 input_report_key(kbd->input, hid_to_key[code], 1)，再查 keymap/shiftmap 生成字符；
2. 上次有、本次没有的 usage 是松开，input_report_key(..., 0)；usage 1 ~ 3 是错误码，两端都跳过；
3. report[0] 的修饰位图逐个和 previous[0] 比较，变化的位用 hid_to_key[224+i]（0xe0 ~ 0xe7）上报按下或松开；
4. 收尾调用 input_sync()，产生一个 EV_SYN/SYN_REPORT，一份报告就是一个完整帧；
5. 普通字符、Backspace、Enter 以及方向键的 ESC [A/B/C/D 转义序列全部送进 consoleintr()，与 UART 收到的字节走同一条路径；
6. 最后 usb_submit_urb() 重新 arm ch6。

字符映射是两张稀疏的表，一张出字符、一张出 Linux 键码：

```
// kernel/usbkbd.c，节选。两张表都按 HID usage 编号索引
static const uchar keymap[58]   = { [4]='a', [16]='m', [30]='1', [40]='\n',
                                    [42]='\b', [43]='\t', [56]='/' };
static const uchar shiftmap[58] = { [4]='A', [16]='M', [30]='!', [45]='_',
                                    [47]='{' };
static const ushort hid_to_key[256] = { [4]=KEY_A, [40]=KEY_ENTER,
                                        [82]=KEY_UP, [225]=KEY_LEFTSHIFT };
```

keymap[]/shiftmap[] 只覆盖能直接打成 ASCII 的 usage 4 ~ 56（字母、数字、常用符号、Enter/Tab/Backspace/Esc/空格）：Shift 决定取哪一张，字母还要再异或一下软件维护的 Caps Lock 状态（usbkbd_key() 里 usage 57 只翻转这个标志，自己不产生字符）；Ctrl 加字母折成 c - 'a' + 1 这样的控制字符，所以 Ctrl-C 就是 0x03。hid_to_key[] 则是 HID usage 到 Linux KEY_* 编号的映射，覆盖普通键、四个方向键和 usage 0xe0 ~ 0xe7 的修饰键；probe() 遍历它一遍，把这台设备支持的键登记成 input 设备的能力位图，于是 evdev 的用户拿到的是与 Linux 一致的编号，而不是 HID 自己的一套。两张表都没登记的 usage——F1 ~ F12、多媒体键——既不产生字